C.V. RAMAN GLOBAL UNIVERSITY

BHUBANESWAR, ODISHA-752054

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FINAL PROJECT REPORT

Analysis and Solution of 30 LeetCode Coding Problems using Python

Guided By: Ajith Kumar Valmiki

NAME:	**Akash Ghadai**
REGD.NO:	**2401030003**
SEMESTER:	**6th Sem**
GROUP:	**B2**

INDEX PAGE

SL. NO.	PROBLEM NAME	LEETCODE PROBLEM NO	PAGE NO	FACULTY SIGN.	REMARKS
1	Two Sum	1	1-2		
2	Longest Substring Without Repeating Characters	3	2-3		
3	Container With Most Water	11	3-4		
4	Remove Duplicates from Sorted Array	26	4-5		
5	Valid Parentheses	20	5-6		
6	Merge Sorted Array	88	6-7		
7	Valid Palindrome	125	7-8		
8	Reverse Words in a String	151	8-9		
9	Min Stack	155	9-10		
10	Contains Duplicate	217	10-11		
11	Implement Queue Using Stacks	232	11-12		
12	Binary Search	704	12-13		
13	First Bad Version	278	13-14		
14	Reverse String	344	14-15		
15	Reverse Vowels of a String	345	15-16		
16	Middle of the Linked List	876	16		
17	Reverse Linked List	206	17		
18	Linked List Cycle	141	18		

SL. NO.	PROBLEM NAME	LEETCODE PROBLEM NO	PAGE NO	FACULTY SIGN.	REMARKS
19	Remove Nth Node From End of List	19	19		
20	Intersection of Two Linked Lists	160	20		
21	Maximum Average Subarray	643	21		
22	Maximum Number of Vowels in Substring	1456	22		
23	Fruit Into Baskets	904	23		
24	Sliding Window Maximum	239	24-25		
25	Koko Eating Bananas	875	25-26		
26	Sort Colors	75	26-27		
27	Validate Binary Search Tree	98	27-28		
28	Binary Tree Level Order Traversal	102	28-29		
29	Number of Islands	200	29-30		
30	Rotting Oranges	994	31-32		

Analysis and Implementation of 30 Leet-Code Problems using Data Structures and Algorithms

1. Two Sum

1. Problem Statement

Find two numbers in a given array that add up to a specific target value and return their positions (indices).

2. Description

We are given an array of integers and an integer representing a target sum. We need to identify exactly two numbers in the array that sum up to this target and return their indices. Key constraints: each input has exactly one valid solution, we cannot use the same element twice, and the order of the returned indices does not matter. This problem represents a fundamental search task where optimizing the lookup of complementary values is key.

3. Algorithm / Approach

To solve this efficiently, we can use a Hash Map (dictionary in Python) to keep track of the numbers we have seen so far and their indices. 1. Initialize an empty dictionary. 2. Iterate through the array using both the index and the current number. 3. For each number, calculate its "complement" (i.e., `target - current number`). 4. Check if this complement already exists in our dictionary. 5. If it does, we have found our two numbers! Return the index of the complement and the current index. 6. If it doesn't, add the current number and its index to the dictionary and continue.

4. Source Code

```python
def twoSum(nums, target):
    # Dictionary to store the value and its index
    seen = {}

    # Iterate through the array
    for i, num in enumerate(nums):
        complement = target - num

        # If complement is found, return the indices
        if complement in seen:
            return [seen[complement], i]

        # Store the current number and its index
        seen[num] = i

    return []
```

5. Input

```python
nums = [2, 7, 11, 15]
target = 9
```

6. Output

```python
[0, 1]
```

7. Conclusion

This problem teaches the usefulness of Hash Maps for fast lookups. Instead of using a nested loop to check all pairs (which takes $O(n^2)$ time), the Hash Map approach reduces the time complexity to **O(n)** and space complexity to **O(n)** by trading memory for speed.

---

2. Longest Substring Without Repeating Characters

1. Problem Statement

Find the length of the longest continuous sequence of characters in a string that does not contain any repeated characters.

2. Description

Given a string, the goal is to find the longest substring where every character is unique. The substring must be contiguous, meaning characters correspond to adjacent positions in the original string. It tests the ability to dynamically track a sequence and shrink or restructure it when a duplicate is encountered.

3. Algorithm / Approach

This problem can be solved using the **Sliding Window** technique with a Hash Set or Hash Map. 1. Use two pointers (`left` and `right`) to represent the boundaries of the current substring (window). 2. Use a dictionary to store the character and its most recent index. 3. Expand the window by moving the `right` pointer. 4. If the character at `right` is already in the dictionary and its index is inside the current window, shrink the window by moving the `left` pointer to `seen[character] + 1`. 5. Update the character's latest index in the dictionary. 6. Calculate the window length (`right - left + 1`) and update the maximum length found so far.

4. Source Code

```python
def lengthOfLongestSubstring(s):
    seen = {}
    max_len = 0
    left = 0

    for right, char in enumerate(s):
        # If we have seen the char and it is in the current window
        if char in seen and seen[char] >= left:
            # Move the left pointer past the duplicate
            left = seen[char] + 1

        # Update the latest index of the character
        seen[char] = right

        # Calculate maximum length
        max_len = max(max_len, right - left + 1)

    return max_len
```

5. Input

```python
s = "abcabcbb"
```

6. Output

```
3  # (The substring is "abc")
```

7. Conclusion

The sliding window technique combined with a Hash Map is incredibly effective for substring problems. Tracking the latest index of characters allows us to jump the `left` pointer directly, making the solution highly optimized. The time complexity is **O(n)** and the space complexity is **O(min(n, m))** where m is the character set size.

---

3. Container With Most Water

1. Problem Statement

Given an array containing the heights of vertical lines drawn on a graph, find two lines that together with the x-axis form a container that holds the maximum amount of water.

2. Description

You are given an integer array `height` representing heights of walls. The distance between two bars is the difference of their indices. You need to calculate the maximum area of water that can be trapped between any two walls. Since water spills over the shorter wall, the height of the container is determined by the shorter of the two walls.

3. Algorithm / Approach

We use the **Two Pointers** approach. 1. Place one pointer at the beginning (`left = 0`) and one at the end (`right = len(height) - 1`). 2. Calculate the area formed by the two lines: `Area = width * height`, where `width = right - left` and `height = min(height[left], height[right])`. 3. Keep track of the maximum area seen so far. 4. Move the pointer pointing to the shorter line inward. Why? Because the width is decreasing, so the only way to possibly get a larger area is to find a taller line. 5. Repeat until the two pointers meet.

4. Source Code

```python
def maxArea(height):
    left, right = 0, len(height) - 1
    max_water = 0

    while left < right:
        # Calculate current area
        current_width = right - left
        current_height = min(height[left], height[right])
        current_area = current_width * current_height

        # Update maximum area
        max_water = max(max_water, current_area)

        # Move the pointer of the shorter bar inward
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return max_water
```

5. Input

```
height = [1,8,6,2,5,4,8,3,7]
```

6. Output

```
49
```

7. Conclusion

This problem showcases a greedy two-pointer strategy. By smartly deciding which wall to discard (the shorter one), we avoid checking all possible pairs, reducing the time complexity from $O(n^2)$ to **O(n)**. The space complexity is **O(1)**.

---

4. Remove Duplicates from Sorted Array

1. Problem Statement

Remove duplicate elements from a sorted array in place, such that each unique element appears only once, and return the total number of unique elements.

2. Description

Given an integer array `nums` sorted in non-decreasing order, the task is to modify the array directly so the first `k` elements contain the unique numbers, maintaining their relative order. You cannot use extra space for another array; the operation must be done in-place. The function should return `k`.

3. Algorithm / Approach

Since the array is sorted, duplicates will be adjacent. We can use the **Two Pointers** technique. 1. Use a pointer `insert_pos` (starting at 1) to track where the next unique element should be placed. 2. Iterate through the array starting from the second element (index 1) with another pointer `i`. 3. Compare the current element `nums[i]` with the previous element `nums[i - 1]`. 4. If they are different, it means we found a new unique element. We place it at `nums[insert_pos]` and increment `insert_pos`. 5. At the end of the loop, `insert_pos` will hold the count of unique elements and the array will be properly modified up to that index.

4. Source Code

```python
def removeDuplicates(nums):
     if not nums:
         return 0

    # insert_pos tracks the position for the next unique element
    insert_pos = 1

    for i in range(1, len(nums)):
        # If we find a new unique element
        if nums[i] != nums[i - 1]:
            nums[insert_pos] = nums[i]
            insert_pos += 1

    return insert_pos
```

5. Input

```
nums = [0,0,1,1,1,2,2,3,3,4]
```

6. Output

```
5  # The modified nums array will be [0,1,2,3,4,...]
```

7. Conclusion

This demonstrates modifying arrays in-place using two pointers: one for reading and one for writing. Relying on the sorted nature of the input makes it remarkably easy to spot duplicates. Time complexity is **O(n)** and space complexity is **O(1)**.

---

5. Valid Parentheses

1. Problem Statement

Check if a string consisting purely of bracket characters (`()`, `{}`, `[]`) is closed in the correct order.

2. Description

We are given a string `s` containing only brackets. A string is considered valid if open brackets are closed by the same type of brackets, and they are closed in the exact correct order. This is a classic parsing problem essential for compilers checking code syntax.

3. Algorithm / Approach

A **Stack** data structure is ideal for this since we want to process the matching properties in a Last-In, First-Out (LIFO) manner. 1. Create an empty stack (a list in Python) and a hash map linking each closing bracket to its corresponding opening bracket. 2. Iterate over each character in the string. 3. If it is an opening bracket, push it onto the stack. 4. If it is a closing bracket, check if the stack is empty. If it is, the string is invalid. 5. If the stack is not empty, pop the top element. If the popped bracket does not match the required opening bracket for the current closing character, it is invalid. 6. After processing all characters, the string is valid only if the stack is empty (meaning all brackets were properly closed).

4. Source Code

```python
def isValid(s):
    stack = []
    # Map closing brackets to their corresponding opening brackets
    mapping = {')': '(', '}': '{', ']': '['}

    for char in s:
        # If it's a closing bracket
        if char in mapping:
            # Pop the top element if stack is not empty, else assign a dummy value
            top_element = stack.pop() if stack else '#'

            # If the popped element doesn't match the corresponding opening bracket, return False
            if mapping[char] != top_element:
                return False
        else:
            # It's an opening bracket, purely push to the stack
            stack.append(char)

    # Valid if stack is empty
    return not stack
```

5. Input

```
s = "([{}])"
```

6. Output

```
True
```

7. Conclusion

The stack abstract data type is a natural fit for problems involving nested structures. Using a dictionary for bracket mappings makes the code extensible and clean. Time complexity is **O(n)** and space complexity is **O(n)**.

---

6. Merge Sorted Array

1. Problem Statement

Merge two sorted integer arrays into one single sorted array.

2. Description

You are given two sorted arrays, `nums1` and `nums2`. `nums1` has enough extra space (filled with zeros) at the end to accommodate `nums2`. The goal is to merge `nums2` directly inside `nums1` in sorted, non-decreasing order without using additional array allocations. `nums1` has `m` active elements, and `nums2` has `n` elements.

3. Algorithm / Approach

To avoid overwriting elements in `nums1` that haven't been processed yet, we utilize a **Reverse Three-Pointer** strategy. 1. Point `p1` to the last valid element of `nums1` (`m - 1`). 2. Point `p2` to the last element of `nums2` (`n - 1`). 3. Point `p` to the very last available slot in `nums1` (`m + n - 1`). 4. Compare `nums1[p1]` and `nums2[p2]`. Place the larger element at `nums1[p]` and decrement `p` and the pointer of the array we picked the element from. 5. If `nums2` elements are leftover after `p1` finishes, copy them over into the remaining slots.

4. Source Code

```python
def merge(nums1, m, nums2, n):
    # Initialize pointers for nums1, nums2, and the merged insertion position
    p1 = m - 1
    p2 = n - 1
    p = m + n - 1

    # Iterate backwards while both arrays have elements left
    while p1 >= 0 and p2 >= 0:
        if nums1[p1] > nums2[p2]:
            nums1[p] = nums1[p1]
            p1 -= 1
        else:
            nums1[p] = nums2[p2]
            p2 -= 1
        p -= 1

    # If nums2 has leftover elements, copy them
    while p2 >= 0:
        nums1[p] = nums2[p2]
        p2 -= 1
        p -= 1
    # Note: No need to check leftover elements in nums1 as they are already in place
```

5. Input

```
nums1 = [1,2,3,0,0,0]
m = 3
nums2 = [2,5,6]
n = 3
```

6. Output

```
# nums1 is modified in-place to:
[1, 2, 2, 3, 5, 6]
```

7. Conclusion

Filling a pre-allocated array starting from the back is a brilliant trick to do in-place sorting without shifting data continuously. Time complexity is strictly **O(m + n)** with **O(1)** space complexity.

---

7. Valid Palindrome

1. Problem Statement

Determine if a given string is a palindrome after removing all non-alphanumeric characters and ignoring uppercase/lowercase differences.

2. Description

A palindrome reads the same forwards and backwards. In this problem, we are given a string that might contain spaces, punctuation marks, and mixed case letters. We must first "clean" the string conceptually by looking only at alphanumeric characters natively case-insensitively, and then verify if the resulting string is symmetrical.

3. Algorithm / Approach

We can use a **Two Pointers** method starting from opposite ends of the string to save space and time. 1. Initialize `left` pointer to the first character and `right` pointer to the last character. 2. Move the `left` pointer continuously forward if it points to a non-alphanumeric character. 3. Move the `right` pointer continuously backward if it points to a non-alphanumeric character. 4. Once both point to valid characters, compare them (converted to lowercase). If they don't match, return `False`. 5. If they match, move both pointers inward. 6. Continue this until `left >= right`. If the loop finishes without issues, return `True`.

4. Source Code

```python
def isPalindrome(s):
    # Initialize two pointers
    left, right = 0, len(s) - 1

    while left < right:
        # Skip non-alphanumeric characters from left
        while left < right and not s[left].isalnum():
            left += 1

        # Skip non-alphanumeric characters from right
        while left < right and not s[right].isalnum():
            right -= 1

        # Compare characters
        if s[left].lower() != s[right].lower():
```

```
            return False

        # Move pointers inward
        left += 1
        right -= 1

    return True
```

5. Input

```
s = "A man, a plan, a canal: Panama"
```

6. Output

```
True
```

7. Conclusion

Two Pointers is highly efficient for validating symmetry because it evaluates conditions simultaneously from both edges. This prevents us from allocating additional memory for a cleaned version of the string. Time complexity is **O(n)** and space complexity is **O(1)**.

---

8. Reverse Words in a String

1. Problem Statement

Reverse the order of words in a given string.

2. Description

Given an input string `s` that may contain leading/trailing spaces or multiple spaces between words, we want to return a string of the words reversed. A word is defined as a sequence of non-space characters. The returned string must only have a single space separating the words, with no leading or trailing spaces.

3. Algorithm / Approach

Python provides built-in methods that make this process straightforward. 1. Use the `split()` function to divide the string into a list of words. This built-in function naturally ignores extra spaces between words. 2. Reverse the list of words. We can do this using the slice syntax `[::-1]` or the `reversed()` function. 3. Join the reversed list back into a single string using `' '.join()`.

4. Source Code

```
def reverseWords(s):
    # split() automatically removes leading/trailing and multiple spaces
    words = s.split()

    # Reverse the array of words
    words.reverse()

    # Join the words with a single space
    return " ".join(words)
```

5. Input

```
s = "  hello world  "
```

6. Output

"world hello"

7. Conclusion

While writing this from scratch (like in C) requires multiple reversals to do it in-place, Python's string manipulation methods abstract the heavy lifting. The time complexity is **O(n)** to parse strings, and the space complexity is **O(n)** to hold the array of words.

---

6. Min Stack

1. Problem Statement

Design a stack that supports standard `push`, `pop`, and `top` operations, but also provides an operation to retrieve the minimum element in constant time.

2. Description

The challenge is managing the current minimum value without having to search the entire stack each time it is requested. The `getMin` function is expected to return the minimal element on the stack immediately in O(1) time. All other stack operations must operate in O(1) time as well.

3. Algorithm / Approach

The solution involves using a standard stack conceptually but keeping track of minimum values alongside real data. 1. Initialize a list `stack`. Instead of just storing the values directly, we'll store tuples (or pairs): `(current_value, current_minimum)`. 2. For `push(val)`: if the stack is empty, the `current_minimum` is `val`. If not, the `current_minimum` is the minimum of `val` and the minimum of the previous top element. We append `(val, min(val, previous_min))` to the stack. 3. For `pop()`: simply pop the top pair from the stack list. 4. For `top()`: return the first item of the pair at the top of the stack. 5. For `getMin()`: return the second item of the pair at the top of the stack.

4. Source Code

```python
class MinStack:
    def __init__(self):
        # We will store tuples of (value, minimum_at_this_level)
        self.stack = []

    def push(self, val: int) -> None:
        if not self.stack:
            self.stack.append((val, val))
        else:
            # The new minimum is the min of the new value and the current minimum
            current_min = self.stack[-1][1]
            self.stack.append((val, min(val, current_min)))

    def pop(self) -> None:
        if self.stack:
            self.stack.pop()

    def top(self) -> int:
        if self.stack:
            return self.stack[-1][0]
```

```python
    def getMin(self) -> int:
        if self.stack:
            return self.stack[-1][1]
```

5. Input

```python
minStack = MinStack()
minStack.push(-2)
minStack.push(0)
minStack.push(-3)
minStack.getMin()
minStack.pop()
minStack.top()
minStack.getMin()
```

6. Output

```python
# Sequential outputs:
# getMin() -> -3
# pop()    -> None
# top()    -> 0
# getMin() -> -2
```

7. Conclusion

Storing the state or "history" of minimums synchronized with elements at every stage is an elegant way to maintain an O(1) retrieval time. Time complexity for all operations is **O(1)**, while space complexity is **O(n)** due to storing pairs instead of just elements.

---

10. Contains Duplicate

1. Problem Statement

Check whether an integer array contains any duplicate values.

2. Description

Given an integer array `nums`, we need to return `True` if any value appears at least twice in the array, and return `False` if every element is distinct. This is a basic frequency/existence check common in database and validation algorithms.

3. Algorithm / Approach

Using a **Hash Set** provides the most optimal time complexity. 1. Initialize an empty Set. Set structures inherently map unique keys and prohibit duplicates. 2. Iterate through each number in `nums`. 3. If the number is already in the set, we've found a duplicate! Return `True`. 4. If not, add the number to the set. 5. If the loop ends without finding duplicates, it means all elements are distinct, so return `False`. *(Alternatively, simply compare the length of the list to the length of the list converted to a set: `len(set(nums)) != len(nums)`).*

4. Source Code

```python
def containsDuplicate(nums):
    # A set to keep track of seen numbers
    seen = set()
```

```python
    for num in nums:
        # Duplicate detected immediately
        if num in seen:
            return True
        seen.add(num)

    return False

# Alternatively, the 1-line Pythonic solution:
# return len(set(nums)) != len(nums)
```

5. Input

```python
nums = [1, 2, 3, 1]
```

6. Output

```
True
```

7. Conclusion

Using a Hash Set restricts duplicate insertion, which makes finding redundant elements O(1) lookup time. The entire array check runs in **O(n)** time. The space complexity is **O(n)** since the set may grow to the size of the array if all elements are unique.

---

11. Implement Queue Using Stacks

1. Problem Statement

Implement a First-In-First-Out (FIFO) queue using only two standard Last-In-First-Out (LIFO) stacks.

2. Description

A standard queue processes elements in a FIFO manner, similar to a line at a grocery store. A stack, however, works as LIFO. The challenge is to combine two stacks (`s1` and `s2`) to mimic the behavior of a queue.

3. Algorithm / Approach

We use two stacks: an `in_stack` to handle pushes, and an `out_stack` to handle pops and peeks. 1. **Push**: Always push newly inserted elements to the `in_stack`. 2. **Pop/Peek**: Since the oldest elements are at the bottom of the `in_stack`, we must move them. If `out_stack` is empty, pop all items from `in_stack` and push them into `out_stack`. 3. Then, simply pop or peek from the `out_stack`. 4. **Empty**: The queue is only empty if both stacks are empty.

4. Source Code

```python
class MyQueue:
    def _init_(self):
        self.in_stack = []
        self.out_stack = []

    def push(self, x: int) -> None:
        self.in_stack.append(x)

    def pop(self) -> int:
        self.move_elements()
        return self.out_stack.pop()
```

```python
    def peek(self) -> int:
        self.move_elements()
        return self.out_stack[-1]

    def empty(self) -> bool:
        return not self.in_stack and not self.out_stack

    def move_elements(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
```

5. Input

```python
q = MyQueue()
q.push(1)
q.push(2)
q.peek()
q.pop()
q.empty()
```

6. Output

```python
# peek() -> 1
# pop() -> 1
# empty() -> False
```

7. Conclusion

Reversing a LIFO structure twice yields a FIFO order. Amortized time complexity for `pop` and `peek` is **O(1)**. Space complexity is **O(n)**.

---

12. Binary Search

1. Problem Statement

Search for a specific target value in a sorted array of integers and return its index.

2. Description

Given an array `nums` sorted in ascending order and a `target` element, we need to return the target's index if it exists, or `-1` if it doesn't. Algorithm must run in O(log n) time.

3. Algorithm / Approach

1. Define two pointers: `left` at the start and `right` at the end.
2. Calculate the middle index `mid`.
3. Compare the item at `mid` to the `target`.
4. If it's a match, return `mid`.
5. If the `target` is greater, update `left = mid + 1`.
6. If the `target` is smaller, update `right = mid - 1`.
7. Repeat until `left` passes `right`. Return `-1` if not found.

4. Source Code

```
def search(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

5. Input

```
nums = [-1,0,3,5,9,12]
target = 9
```

6. Output

```
4
```

7. Conclusion

Binary search is a cornerstone concept illustrating Divide and Conquer. Time complexity is **O(log n)**, and space complexity is **O(1)**.

---

13. First Bad Version

1. Problem Statement

Find the first broken version in a product's continuous version history.

2. Description

You are given an API `isBadVersion(version)` that checks if a version is bad. All versions after a bad version are also bad. Find the first bad version minimizing API calls.

3. Algorithm / Approach

1. Initialize markers: `left = 1` and `right = n`.
2. Compute `mid`. Query `isBadVersion(mid)`.
3. If it is bad, `mid` *might* be the first bad version. Move the upper limit: `right = mid`.
4. If `mid` is not bad, move the lower limit: `left = mid + 1`.
5. Terminate when `left` meets `right`. `left` will end on the first bad version.

4. Source Code

```
# def isBadVersion(version: int) -> bool:

def firstBadVersion(n):
    left = 1
    right = n
```

```
    while left < right:
        mid = left + (right - left) // 2

        if isBadVersion(mid):
            right = mid
        else:
            left = mid + 1

    return left
```

5. Input

```
n = 5
# bad = 4
```

6. Output

```
4
```

7. Conclusion

An application of binary search looking for a Boolean boundary. Time complexity is **O(log n)** and space complexity **O(1)**.

---

14. Reverse String

1. Problem Statement

Reverse an array of strings directly in-place.

2. Description

Given an array of characters, reverse the element order without generating a secondary array. Modify the list sequentially using its original memory.

3. Algorithm / Approach

1. Place a pointer `left` at the 0th index, and `right` at the last index.
2. Swap the character at `left` with the character at `right`.
3. Move `left` forward (+1) and `right` backwards (-1).
4. Exit the loop when `left` meets or passes `right`.

4. Source Code

```
def reverseString(s):
    left, right = 0, len(s) - 1

    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1
```

5. Input

```
s = ["h","e","l","l","o"]
```

6. Output

```
["o","l","l","e","h"]
```

7. Conclusion

Array modification with two pointers on the ends converging toward the middle is an **O(n)** time mechanism utilizing an **O(1)** overhead footprint.

---

15. Reverse Vowels of a String

1. Problem Statement

Reverse only the vowel characters inside a given string.

2. Description

Swap only the vowels (a, e, i, o, u) present in the string. Both uppercase and lowercase variants matter.

3. Algorithm / Approach

1. Convert string to a list of characters.
2. Start `left` at the beginning and `right` at the end.
3. Move `left` forward until it hits a vowel, and `right` backward until it hits a vowel.
4. Swap elements at `left` and `right`.
5. Move both pointers inward and repeat until they meet.
6. Reconstruct the list into a string and return.

4. Source Code

```python
def reverseVowels(s):
    vowels = set("aeiouAEIOU")
    s_list = list(s)
    left, right = 0, len(s_list) - 1

    while left < right:
        while left < right and s_list[left] not in vowels:
            left += 1
        while left < right and s_list[right] not in vowels:
            right -= 1
        if left < right:
            s_list[left], s_list[right] = s_list[right], s_list[left]
            left += 1
            right -= 1

    return "".join(s_list)
```

5. Input

```
s = "leetcode"
```

6. Output

```
"leotcede"
```

7. Conclusion

Using a set to evaluate character typings alongside two pointers maintains maximum efficiency. Time complexity stays **O(n)** with space at **O(n)** handling string immutability.

---

16. Middle of the Linked List

1. Problem Statement

Find and return the middle node of a singly linked list.

2. Description

Given the head of a singly linked list, return the halfway point structure. If the list has an even amount of components, return the second middle node.

3. Algorithm / Approach

1. Initiate two pointers, `slow` and `fast`, both starting at the `head`.
2. `fast` moves twice as fast as `slow` (two nodes per iteration).
3. `slow` traverses one node per iteration.
4. When `fast` reaches the end of the list, `slow` will be exactly at the midway point.

4. Source Code

```
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

def middleNode(head):
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    return slow
```

5. Input

```
head = [1, 2, 3, 4, 5]
```

6. Output

```
[3, 4, 5]
```

7. Conclusion

Calculations boil down to a single iteration path without relying on total lengths, providing strictly **O(n)** runtimes and **O(1)** memory mapping.

17. Reverse Linked List

1. Problem Statement

Reverse a singly linked list completely and supply the new head.

2. Description

Restructure the `next` pointers within the nodes so that the list flows backwards. Return the node that was previously the tail.

3. Algorithm / Approach

1. Use variables: `prev` (None), `curr` (head).
2. For each node, temporarily save the next node.
3. Bind the current node's pointer backward to `prev`.
4. Shift markers: `prev` becomes `curr`, and `curr` slides into the saved next node.
5. `prev` will eventually represent the new head.

4. Source Code

```
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

def reverseList(head):
    prev = None
    curr = head

    while curr:
        next_temp = curr.next
        curr.next = prev
        prev = curr
        curr = next_temp

    return prev
```

5. Input

```
head = [1, 2, 3, 4, 5]
```

6. Output

```
[5, 4, 3, 2, 1]
```

7. Conclusion

Incrementally reversing pointers handles linking robustly in **O(n)** time and **O(1)** space logic.

---

18. Linked List Cycle

1. Problem Statement

Detect if a continuous linked list contains an internal circular loop.

2. Description

Determine if a singly linked list has a cycle, meaning the tail points to a node previously visited instead of `None`. Return `True` or `False`.

3. Algorithm / Approach

1. Use Floyd's Cycle-Finding Algorithm. Place `slow` and `fast` pointers at the `head`.
2. Advance `slow` by 1, and `fast` by 2.
3. If no cycle exists, `fast` encounters `None`.
4. If a loop wraps round, `fast` will recursively loop and collide with `slow`. Check if their references match.

4. Source Code

```
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

def hasCycle(head):
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

        if slow == fast:
            return True

    return False
```

5. Input

```
head = [3, 2, 0, -4]
pos = 1  # -4 points to 2
```

6. Output

```
True
```

7. Conclusion

Mathematical tracking of variable speeds avoids massive object mapping tables. Time complexity **O(n)** and Memory complexity strictly **O(1)**.

16. Remove Nth Node From End of List

1. Problem Statement

Remove the nth node from the end of a linked list and recover the updated list's head.

2. Description

Complete the node deletion counting from the tail threshold in exactly 1-pass since linked lists traverse solely forward natively.

3. Algorithm / Approach

1. Instantiate a "dummy" node ahead of the head.
2. Initialize `first` and `second` pointers at dummy.
3. Propel `first` ahead structurally exactly $n + 1$ instances.
4. Move both pointers at equal paces until `first` surpasses the chain end.
5. The pre-calculated spacing drops `second` exactly prior to the target node. `second.next = second.next.next`.

4. Source Code

```
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

def removeNthFromEnd(head, n):
    dummy = ListNode(0)
    dummy.next = head
    first = dummy
    second = dummy

    for _ in range(n + 1):
        first = first.next

    while first is not None:
        first = first.next
        second = second.next

    second.next = second.next.next

    return dummy.next
```

5. Input

```
head = [1, 2, 3, 4, 5]
n = 2
```

6. Output

```
[1, 2, 3, 5]
```

7. Conclusion

Enabling fixed separation windows eliminates secondary counting logic. Runtime **O(n)** combined cleanly with **O(1)** memory constants.

20. Intersection of Two Linked Lists

1. Problem Statement

Locate the precise node where two distinct one-directional linked branches merge together.

2. Description

Given isolated list bases `headA` and `headB`, deliver the node reference where intersection transpires physically in memory. Unmerged outputs must present empty `None`.

3. Algorithm / Approach

1. Create `ptrA` starting at `headA` mapping beside `ptrB` starting at `headB`.
2. Simultaneously iterate steps.
3. When either hits `None`, forcefully restart it from the opposite list's base.
4. By swapping paths, both pointers travel effectively length(A)+length(B). Therefore, they align concurrently matching intersections.

4. Source Code

```
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

def getIntersectionNode(headA, headB):
    if not headA or not headB:
        return None

    ptrA = headA
    ptrB = headB

    while ptrA != ptrB:
        ptrA = headB if ptrA is None else ptrA.next
        ptrB = headA if ptrB is None else ptrB.next

    return ptrA
```

5. Input

```
# List A: 4 -> 1 \
#                 -> 8 -> 4 -> 5
# List B: 5 -> 6 /
```

6. Output

```
# Node with value 8
```

7. Conclusion

Symmetrical trajectory offsets perfectly match unequal length branches mathematically guaranteeing collision points. Time scales **O(N+M)** using **O(1)** memory.

21. Maximum Average Subarray I

1. Problem Statement

Find a contiguous subarray of exactly length `k` that has the maximum average value, and return this maximum average.

2. Description

Given an integer array `nums` and an integer `k`, we need to find the `k`-length continuous slice of the array whose elements sum up to the highest possible value. Because the length `k` is constant, finding the maximum sum directly translates to finding the maximum average.

3. Algorithm / Approach

This problem is perfectly suited for the **Fixed Sliding Window** technique. 1. Compute the sum of the first `k` elements to form our initial window. 2. Initialize `max_sum` with this initial sum. 3. Slide the window one element to the right at a time across the rest of the array. 4. To update the sum in O(1) time, add the new element entering the window from the right, and subtract the old element leaving the window from the left. 5. Continuously update `max_sum`. 6. Finally, return `max_sum / k` as the maximum average.

4. Source Code

```python
def findMaxAverage(nums, k):
    # Calculate sum of the first `k` elements
    current_sum = sum(nums[:k])
    max_sum = current_sum

    # Slide the window from index k to the end
    for i in range(k, len(nums)):
        # Add the new right element, subtract the old left element
        current_sum += nums[i] - nums[i - k]

        if current_sum > max_sum:
            max_sum = current_sum

    return max_sum / k
```

5. Input

```python
nums = [1,12,-5,-6,50,3]
k = 4
```

6. Output

```python
12.75  # The subarray is [12, -5, -6, 50], sum = 51, avg = 12.75
```

7. Conclusion

The Fixed Sliding Window transforms an O(N*K) brute-force summation into an **O(N)** optimized pass. Space complexity remains **O(1)**.

22. Maximum Number of Vowels in a Substring of Given Length

1. Problem Statement

Find the maximum number of vowel letters present in any substring of a specific length `k`.

2. Description

Given a string `s` and an integer `k`, evaluate every `k`-length contiguous block to see which one contains the highest count of vowels ('a', 'e', 'i', 'o', 'u'). Return that maximum count.

3. Algorithm / Approach

We apply the **Fixed Sliding Window** algorithm similarly to the maximum average subarray. 1. Establish a set of vowels for rapid O(1) condition checking. 2. Form the first window of size `k` and count the exact number of vowels inside it. 3. Record this initial count as our `max_vowels`. 4. Slide the window forward by doing two things: - If the new character entering the window is a vowel, increment the count. - If the character left behind outside the window was a vowel, decrement the count. 5. Maximize the count tracking variable dynamically.

4. Source Code

```python
def maxVowels(s, k):
    vowels = {'a', 'e', 'i', 'o', 'u'}
    current_vowels = 0

    # Count vowels in the first window
    for i in range(k):
        if s[i] in vowels:
            current_vowels += 1

    max_vowel_count = current_vowels

    # Slide the window
    for i in range(k, len(s)):
        # Subtract if leaving character is a vowel
        if s[i - k] in vowels:
            current_vowels -= 1
        # Add if entering character is a vowel
        if s[i] in vowels:
            current_vowels += 1

        max_vowel_count = max(max_vowel_count, current_vowels)

    return max_vowel_count
```

5. Input

```python
s = "abciiidef"
k = 3
```

6. Output

```python
3  # Substring "iii" contains 3 vowels
```

7. Conclusion

By carrying forward state evaluations instead of recalculating counts, time complexity becomes strictly **O(N)**. Memory allocation is **O(1)** since our vowel lookup set holds exactly 5 constant characters.

23. Fruit Into Baskets

1. Problem Statement

Find the longest contiguous stretch of trees that contains no more than two different types of fruit.

2. Description

Given an array `fruits` representing fruit types, you have only two baskets, and each basket can hold an infinite amount of a single type of fruit. You must collect fruits contiguously. Thus, the problem translates to: *Find the length of the longest subarray with at most two distinct elements.*

3. Algorithm / Approach

This utilizes a **Dynamic Sliding Window** aided by a Hash Map. 1. Use `left` and `right` pointers to represent the basket collection bounds. 2. Use a dictionary `basket` to track the frequency of each fruit type in the current window. 3. Expand `right`. If a new fruit exceeds our limit of 2 distinct types, we must shrink the window from the `left`. 4. To shrink, decrement the frequency of `fruits[left]`. If a fruit count reaches 0, delete it from the dictionary. 5. Record the maximum size of the valid window (`right - left + 1`).

4. Source Code

```python
def totalFruit(fruits):
    basket = {}
    max_fruits = 0
    left = 0

    for right in range(len(fruits)):
        # Add current fruit to basket
        fruit = fruits[right]
        basket[fruit] = basket.get(fruit, 0) + 1

        # If we have more than 2 types of fruits, shrink window
        while len(basket) > 2:
            left_fruit = fruits[left]
            basket[left_fruit] -= 1

            # Remove fruit type completely if count reaches 0
            if basket[left_fruit] == 0:
                del basket[left_fruit]
            left += 1

        max_fruits = max(max_fruits, right - left + 1)

    return max_fruits
```

5. Input

```python
fruits = [1,2,3,2,2]
```

6. Output

```python
4  # Subarray is [2,3,2,2]
```

7. Conclusion

Hash Maps paired with a flexible window beautifully resolve constraint-based subarray tracking. While the `while` loops intuitively suggest slower bounds, every fruit is visited strictly twice, providing **O(N)** execution time. Space complexity evaluates to **O(1)** as the dictionary caps at 3 elements.

24. Sliding Window Maximum

1. Problem Statement

Find the maximum element within every sliding window of size `k` moving from left to right across an array.

2. Description

For an array `nums` and window length `k`, you can only see `k` numbers horizontally at a time. The window slides +1 continuously. You must return an array compiling every maximum value observable in that window per step.

3. Algorithm / Approach

Extracting maxes per frame without hitting O(N*K) requires a **Monotonic Deque** (Double-Ended Queue). 1. The queue will store only array* indices*, keeping the actual array values monotonically decreasing. 2. For each element `nums[i]`, remove indices from the back of the queue if their corresponding values are smaller than `nums[i]`. (They can never be the maximum because `nums[i]` is both bigger and will stay in the window longer). 3. Append index `i` to the back. 4. Remove the front index if it has slid out of the window (`front <= i - k`). 5. After hitting length `k`, record the front of the queue, as it consistently represents the highest value.

4. Source Code

```python
from collections import deque

def maxSlidingWindow(nums, k):
    # Store indices, not values
    q = deque()
    result = []

    for i in range(len(nums)):
        # Remove elements out of window boundary
        if q and q[0] == i - k:
            q.popleft()

        # Remove smaller elements in k range as they are useless
        while q and nums[q[-1]] < nums[i]:
            q.pop()

        q.append(i)

        # Window reached size k, add max to result
        if i >= k - 1:
            result.append(nums[q[0]])

    return result
```

5. Input

```python
nums = [1,3,-1,-3,5,3,6,7]
k = 3
```

6. Output

```python
[3, 3, 5, 5, 6, 7]
```

7. Conclusion

Monotonically decreasing deques provide the perfect buffer system filtering obsolete values while actively ordering active ones. Items are aggressively popped keeping constraints tight. Runtime evaluates heavily optimized at **O(N)**. Space complexity relies on **O(K)** holding the active window bounds.

---

25. Koko Eating Bananas

1. Problem Statement

Determine the minimum banana eating speed `k` (bananas per hour) that allows eating all bananas across all piles within `h` hours.

2. Description

Koko wants to eat all `piles` of bananas within `h` hours. She decides her eating speed `k`. Every hour she chooses a pile and eats `k` bananas. If the pile has fewer than `k` bananas, she eats them and rests for the remainder of the hour. What is her minimum necessary speed?

3. Algorithm / Approach

This represents **Binary Search on Answer Space**. 1. Minimum speed `left` is `1`. Maximum speed `right` is `max(piles)`. 2. Binary search the optimal `k` speed. Compute a `mid` speed. 3. Calculate the required hours to finish all piles mapping `math.ceil(pile / mid)`. 4. If total hours required `> h`, she's eating too slowly. Boost speed: `left = mid + 1`. 5. If total hours required `<= h`, she can finish, but we want the minimum speed. Attempt lower bounds: `right = mid`. 6. Terminate resolving `left` as the optimal valid speed.

4. Source Code

```python
import math

def minEatingSpeed(piles, h):
    # Search domain for speeds
    left = 1
    right = max(piles)

    while left < right:
        mid_speed = left + (right - left) // 2

        # Calculate time taken at mid_speed
        hours_taken = sum(math.ceil(pile / mid_speed) for pile in piles)

        if hours_taken > h:
            # We took too long, eat faster
            left = mid_speed + 1
        else:
            # We made it! But can we eat slower?
            right = mid_speed

    return left
```

5. Input

```python
piles = [3,6,7,11]
h = 8
```

6. Output

```
4
```

7. Conclusion

Instead of strictly hunting arrays, searching algorithmic outcome potential spaces proves phenomenally powerful. Runtime scales **O(N log M)** where N matches piles counting and M equals maximum pile limits. Space checks in exclusively at **O(1)**.

---

26. Sort Colors

1. Problem Statement

Sort an array holding elements identically valued purely at 0, 1, and 2 directly in-place without utilizing native library sort functions.

2. Description

You receive an array `nums` mapping red, white, and blue using identifiers 0, 1, and 2 respectively. Group elements physically sorted scaling 0 to 2 optimally requiring 1 solitary pass.

3. Algorithm / Approach

Apply the **Dutch National Flag Algorithm** via threetracking bounds. 1. Track trackers: `low = 0` (boundary for 0s), `mid = 0` (iterator), `high = len(nums) - 1` (boundary for 2s). 2. Sweep `mid` forward evaluating indices until breaking past `high`. 3. If `nums[mid] == 0`: Swap logic positions tying `nums[low]` and `nums[mid]`. Elevate both `low` and `mid`. 4. If `nums[mid] == 1`: Position is already correctly placed centrally. Just push `mid` ahead. 5. If `nums[mid] == 2`: Swap logic bridging `nums[mid]` backwards alongside `nums[high]`. Downgrade `high`, keeping `mid` frozen so the newly swapped-in element can be verified next iteration.

4. Source Code

```python
def sortColors(nums):
    low, mid = 0, 0
    high = len(nums) - 1

    while mid <= high:
        if nums[mid] == 0:
            # Swap 0 to the front partition
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            # 1 belongs in the middle
            mid += 1
        else:
            # Swap 2 to the back partition
            nums[high], nums[mid] = nums[mid], nums[high]
            high -= 1
```

5. Input

```
nums = [2,0,2,1,1,0]
```

6. Output

```
[0,0,1,1,2,2]
```

7. Conclusion

Multi-pointer anchoring isolates partitions effectively pushing numbers cleanly directly via concurrent sorting swaps. Delivering true **O(N)** evaluation using **O(1)** in-place overheads perfectly replicates optimal low-level sorting parameters.

---

27. Validate Binary Search Tree

1. Problem Statement

Check whether a configured Binary Tree is fundamentally structured as a valid Binary Search Tree (BST).

2. Description

In a valid BST, the left subtree contains only values smaller than the parent node, and the right subtree exclusively houses figures fundamentally greater. Both subtrees independently must also recursively represent true BST formats. A standalone DFS check verifies legality.

3. Algorithm / Approach

Validation succeeds leveraging **Recursive Depth-First Search Configuration Bounds**. 1. Create a sub-function establishing parameters feeding current node mappings, absolute `lower` boundary mappings, and `upper` boundary limitations. 2. Initialize starting with unlimited bounding (`-infinity`, `+infinity`). 3. If traversing reaches the end node (`None`), structurally validate returning `True`. 4. If current values break outside lower/upper limitations, fail out providing `False`. 5. Recurse down branching into the left child tightly restricting the upper threshold limit strictly to `current node value`. 6. Recurse heavily into the right child elevating the root lower restriction to `current node value`.

4. Source Code

```python
# class TreeNode:
#     def _init_(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def isValidBST(root):
    def validate(node, low=-float('inf'), high=float('inf')):
        # Empty nodes are valid by default
        if not node:
            return True

        # The current node's value must be strictly within the range
        if node.val <= low or node.val >= high:
            return False

        # The left subtree must be less than node.val
        # The right subtree must be greater than node.val
         return (validate(node.left, low, node.val) and
                 validate(node.right, node.val, high))

    return validate(root)
```

5. Input

```
# Tree maps: [2, 1, 3] (2 at root, 1 left, 3 right)
```

6. Output

```
True
```

7. Conclusion

Tracking constraint parameters iteratively across lower branching enforces complete validity ensuring deeply nested nodes comply strictly with master parent inheritance rules. Calculates **O(N)** scaling traversing structures via **O(N)** internal memory framing limits tracking structural deepness.

---

28. Binary Tree Level Order Traversal

1. Problem Statement

Construct an array arraying the level-by-level (top to bottom, left to right) progression sequence values of a given Binary Tree.

2. Description

Provided a root of a structural binary tree, extract row listings grouping siblings equally aligned geometrically. Represent formats inside 2D-Arrays listing levels progressively. Assures horizontal tracking rather than deep delving mechanisms.

3. Algorithm / Approach

Operate horizontal scanning effectively via **Breadth-First Search (BFS) combined with Queue mechanics**. 1. Setup a list queue holding root structures. Initializations fail if tree references equal `None`. 2. Extract the amount of length existing natively within the current queue representing level size parameters. 3. Iteratively loop removing structures identically bound resolving the precise current size amount previously tracked. 4. Record their inherent values internally per level. 5. Identify left and right branching structures adding them heavily sequentially queuing backwards. 6. Push level results pushing master responses arrays.

4. Source Code

```python
from collections import deque

# class TreeNode:
#     def _init_(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def levelOrder(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        current_level = []
```

```python
        # Process all nodes currently in the queue (one level)
        for _ in range(level_size):
            node = queue.popleft()
            current_level.append(node.val)

            # Add child nodes to the queue for the next level
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        # Append the processed level to the final result
        result.append(current_level)

    return result
```

5. Input

```
# Tree maps: [3, 9, 20, null, null, 15, 7]
```

6. Output

```
[[3], [9, 20], [15, 7]]
```

7. Conclusion

Level grouping isolates row architectures dynamically capturing dimensions uniformly. Leveraging a FIFO queue enforces breadth iterations effectively without risking recursion depths. Traversing scales efficiently locking inside **O(N)** intervals while queue holding takes **O(N)** constraints spanning bottom layers.

---

26. Number of Islands

1. Problem Statement

Evaluate a grid computing raw map layouts establishing "1" (land) and "0" (water). Trace exact island structural formations rendering counts isolating unique contiguous lands.

2. Description

An island encompasses isolated horizontal or vertical "1" cells bounded securely strictly inside native water constraints or array boundaries. Diagnosing configurations demands complete map navigation ensuring connected blocks merge grouping into a solitary island entity count definitively.

3. Algorithm / Approach

Implement intensive **Depth-First Search (DFS) Traversal tracking constraints**. 1. Navigate evaluating grid matrix points exhaustively counting nodes. 2. If identifying unvisited land ('1'), increase maximum island counts. 3. Deploy DFS logic sweeping completely traversing out capturing contiguous formatting. 4. Convert visited lands securely assigning strings formatting ('0') preventing infinite iteration redundancies. 5. DFS systematically tests Up, Down, Left, Right directional bounds sweeping arrays correctly identifying structural land blocks iteratively grouping matches.

4. Source Code

```
def numIslands(grid):
     if not grid:
          return 0

    rows = len(grid)
    cols = len(grid[0])
    island_count = 0

    def dfs(r, c):
        # Base cases to stop DFS: Out of bounds or current cell is water
        if (r < 0 or c < 0 or r >= rows or
            c >= cols or grid[r][c] == "0"):
            return

        # Mark as visited by sinking the island
        grid[r][c] = "0"

        # Look recursively in all 4 directions
        dfs(r + 1, c) # Down
        dfs(r - 1, c) # Up
        dfs(r, c + 1) # Right
        dfs(r, c - 1) # Left

    for r in range(rows):
        for c in range(cols):
            # We found an unvisited part of a new island
            if grid[r][c] == "1":
                island_count += 1
                dfs(r, c) # Sink all connected tiles of this island

    return island_count
```

5. Input

```
grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
```

6. Output

```
3
```

7. Conclusion

"Sinking" land natively effectively functions mapping visited metrics heavily bypassing external matrices significantly saving overhead. Operations span matrix size strictly establishing **O(rows * cols)** temporal constraints alongside equally sized worst-case implicit memory recursions.

30. Rotting Oranges

1. Problem Statement

Gauge matrix orange crop infections analyzing time progression. Calculate completely how many exact minutes expire decaying entire configurations given specific infectious rules.

2. Description

The grid maps 0 (empty), 1 (fresh), and 2 (rotten). Each sequential minute, fresh oranges orthogonally located neighboring rotting units infect deteriorating recursively. Trace formats verifying maximum bounds or evaluating structurally blocked units guaranteeing impossible infections tracking isolated layouts (return -1).

3. Algorithm / Approach

Executesynchronous grid sweeping via **Multi-Source Breadth-First Search**.

1. Formulate counting fresh oranges verifying layout bases and map initially rotting oranges pushing strictly into tracking queues.

2. Traverse dynamically tracking minute counts per iteration layers pulling queued items safely identifying coordinate positioning lengths.

3. Validate directional formatting coordinates spanning Up, Down, Left, and Right locations organically inside matrix lengths mutating '1's heavily converting correctly updating variables tracking fresh losses efficiently back queuing components.

4. When tracking ceases completely verify configurations determining if `fresh_oranges == 0`. Return elapsed minutes safely.

4. Source Code

```
from collections import deque

def orangesRotting(grid):
    rows = len(grid)
    cols = len(grid[0])
    queue = deque()
    fresh_oranges = 0

    # Step 1: Collect all initial rotten oranges and count fresh ones
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c))
            elif grid[r][c] == 1:
                fresh_oranges += 1

    # If no fresh oranges to begin with
    if fresh_oranges == 0:
        return 0

    minutes = 0
    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

    # Step 2: BFS radiating outward from all rotten oranges
    while queue and fresh_oranges > 0:
        minutes += 1
```

```
        # Process everything currently in the queue (one minute pass)
        for _ in range(len(queue)):
            r, c = queue.popleft()

            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                # If valid and fresh, decay it
                if (0 <= nr < rows and 0 <= nc < cols
                   and grid[nr][nc] == 1):
                    grid[nr][nc] = 2 # Mark as rotten
                    fresh_oranges -= 1
                    queue.append((nr, nc))

    # Step 3: Check if we successfully decayed all oranges
    return minutes if fresh_oranges == 0 else -1
```

5. Input

```
grid = [[2,1,1],[1,1,0],[0,1,1]]
```

6. Output

```
4
```

7. Conclusion

Deploying multi-source layouts effectively establishes radiating wavefronts tracking real-world physical spreads. Time progresses sequentially resolving metrics bounding completely. Traversals evaluate cleanly executing within limits capping tightly into **O(rows * cols)** constraints simultaneously mirroring equivalent space bounds.

SUBMITTED BY:

NAME: Akash Ghadai

SEM: 6TH
GROUP: B2

BRANCH: CS&IT

REGD NO: 2401030003